

HW1 – Analytical performance model of SmallCNN

input $3 \times S \times S$, batch B , FP32, eval(), inference_mode, cudnn.benchmark = False, no TF32

0. Shapes. Every stride-2 layer halves the resolution. Activation sizes in units of BS^2 elements (= channels / cumulative stride²):

x : 3 conv1 (7x7 s2, 3->32): $32/4 = 8$ pool1 (3x3 s2): $32/16 = 2$
conv2 (5x5, 32->64): $64/16 = 4$ conv3 (3x3 s2, 64->128): $128/64 = 2$ conv4 (1x1, 128->256): $256/64 = 4$
conv5 (3x3 s2, 256->256): $256/256 = 1$ conv6 (1x1, 256->512): $512/256 = 2$ GAP: $512B$, fc1: $256B$, fc2: $100B$

Parameters: conv = $3 \cdot 32 \cdot 49 + 32 \cdot 64 \cdot 25 + 64 \cdot 128 \cdot 9 + 128 \cdot 256 + 256 \cdot 256 \cdot 9 + 256 \cdot 512$

= $4704 + 51200 + 73728 + 32768 + 589824 + 131072 = 883\,296$

BN: $2 \times 1248 = 2496$ affine + 2496 running stats; fc: $512 \cdot 256 + 256 + 256 \cdot 100 + 100 = 157\,028$

total = $1\,042\,820$ params + 2502 buffers $\Rightarrow W = 4.18$ MB in FP32.

1. FLOPs (convention: 1 MAC = 2 FLOPs).

A conv $C_{in} \rightarrow C_{out}$, kernel k , output $H_o W_o$ does $\text{MACs} = B C_{out} H_o W_o C_{in} k^2$,

with $H_o W_o = S^2 / \sigma^2$, σ = cumulative stride at the output. Per BS^2 :

conv1: $32 \cdot 3 \cdot 49 / 4 = 1176$ conv2: $64 \cdot 32 \cdot 25 / 16 = 3200$ conv3: $128 \cdot 64 \cdot 9 / 64 = 1152$
conv4: $256 \cdot 128 / 64 = 512$ conv5: $256 \cdot 256 \cdot 9 / 256 = 2304$ conv6: $512 \cdot 256 / 256 = 512$

$\Sigma = 8856$ MAC $\Rightarrow \text{FLOPs}_{\text{conv}} = 17\,712 BS^2$.

Minor terms: BN in eval is $y = \gamma'x + \beta'$: 2 FLOP/elem $\rightarrow 2(8+4+2+4+1+2) = 42$; ReLU 1 FLOP/elem $\rightarrow 21$;

maxpool 8 comparisons/output $\rightarrow 16$; GAP 1 add/elem $\rightarrow 2$. Sum = $81 BS^2$.

Head (per image): $2(512 \cdot 256 + 256 \cdot 100) + 256$ (ReLU) + 512 (GAP divide) = $314\,112$.

$$\text{FLOPs}(S, B) = 17\,793 B S^2 + 314\,112 B$$

99.5 % of it is in the six convolutions; $\propto BS^2$ - doubling S quadruples the work, doubling B doubles it.

Check: $S = 224, B = 1$: 0.893 GFLOP; $S = 512, B = 256$: 1.19 TFLOP. (torch.profiler with_flops counts

only conv + linear: $17\,712 BS^2 + 313\,344 B$ - matches exactly.)

2. Memory = peak of `torch.cuda.max_memory_allocated()` during one forward pass.

inference_mode: no autograd graph, nothing is saved for backward. `nn.Sequential` runs `input = m(input)`, so during op l the live tensors are: weights W , the caller's x (alive for the whole pass), the op's input and its freshly allocated output. In-place ReLU allocates nothing; Flatten is a view.

Live activations per op, in BS^2 elements ($x = 3$ always included):

conv1: $3 + 8 = 11$ **bn1**: $3 + 8 + 8 = 19$ relu1: 11 pool1: $3 + 8 + 2 = 13$
conv2: $3 + 2 + 4 = 9$ bn2: $3 + 4 + 4 = 11$ conv3: 9 bn3: 7 conv4: 9 bn4: 11
conv5: 8 bn5: 5 conv6: 6 bn6: 7 gap: 5 (all ≤ 19)

The peak is inside **bn1**: x , the conv1 output and the bn1 output coexist: $19BS^2$ floats = $76BS^2$ bytes.

Constant part, resident for the whole pass: parameters + buffers 4.18 MB, plus PyTorch's cuBLAS workspaces, allocated through the caching allocator at the first GEMM (fc1) and never freed:

cuBLAS (CUBLAS_WORKSPACE_CONFIG default "4096:2:16:8"): $4096 \cdot 2 + 16 \cdot 8 = 8320$ KiB,
cuBLASLt (CUBLASLT_WORKSPACE_SIZE default): 1024 KiB $\Rightarrow$ 9.57 MB. Total constant: 13.75 MB.

$$\text{Memory}(S, B) = 76 BS^2 + 13.75 \times 10^6 \text{ bytes}$$

Assumptions / not modelled: cuDNN convolution workspace. With `cudnn.benchmark = False` the heuristic may pick, for small problems, engines that need 5-70 MiB of scratch (observed for conv2, 5×5). It only matters while $76BS^2$ is itself small ($BS^2 \leq 10^5$). Allocator rounds every block up to 512 B (negligible). OOM is predicted when $\text{Memory}(S, B)$ exceeds the free device memory.

3. Bytes moved (ideal traffic: each activation is written once by its producer and read once by its consumer, weights read once; every PyTorch op is a separate kernel, so no fusion and no reuse in L2 assumed).

conv1: $3+8$ bn1: $8+8$ relu1: $8+8$ pool1: $8+2$ conv2: $2+4$ bn2, relu2: $2 \cdot (4+4)$
conv3: $4+2$ bn3, relu3: $2 \cdot (2+2)$ conv4: $2+4$ bn4, relu4: $2 \cdot (4+4)$
conv5: $4+1$ bn5, relu5: $2 \cdot (1+1)$ conv6: $1+2$ bn6, relu6: $2 \cdot (2+2)$ gap: 2
 $\Sigma = 133BS^2$ elements; head: $512 + (512+256) + (256+256) + (256+100) = 2148$ per image.

$$\text{Bytes}(S, B) = 4(133 BS^2 + 2148 B) + W_{\text{bytes}} \approx 532 BS^2 + 8592 B + 4.18 \times 10^6$$

Arithmetic intensity of the whole net: $17793/532 \approx 33$ FLOP/byte, close to the RTX 3090 ridge point 35.6 TFLOP/s / 936 GB/s = 38 FLOP/byte. Per layer it is very uneven:

conv2: $6400/24 = 267$, conv5: $4608/20 = 230$, conv1: $2352/44 = 53$ (compute-bound);
BN: $2/8 = 0.25$, ReLU: $1/8$, pool: $16/40$ (memory-bound, 100-300x below the ridge).

4. Latency (median wall time of one forward pass; eval + inference_mode, FP32, cudnn.benchmark = False).

Assumptions: (a) kernels run back-to-back on one stream, layers do not overlap;
(b) each of the $N = 23$ kernel-launching ops costs at least t_0 - eager-mode dispatch + launch on the CPU;
the GPU idles while it waits, so t_0 is a floor, not an additive term;
(c) inside a kernel, DRAM traffic and arithmetic overlap perfectly, so the kernel time is the larger of the two
(roofline); (d) one effective bandwidth β and one effective FP32 throughput π for all kernels;
(e) stationary clocks/temperature (warm-up before timing).

$$\mathbf{T}(\mathbf{S}, \mathbf{B}; \boldsymbol{\theta}) = \sum_{\ell=1}^N \max\left(t_0, \frac{\text{Bytes}_{\ell}(\mathbf{S}, \mathbf{B})}{\beta}, \frac{\text{FLOPs}_{\ell}(\mathbf{S}, \mathbf{B})}{\pi}\right), \quad \boldsymbol{\theta} = (t_0, \beta, \pi)$$

with $\text{FLOPs}_{\ell} = f_{\ell} BS^2$ and $\text{Bytes}_{\ell} = 4(a_{\ell} BS^2 + w_{\ell})$ from the tables above.

Regimes as BS^2 grows:

launch-bound: $T \approx Nt_0$ while even the largest layer has $\text{Bytes}_{\ell}/\beta < t_0$, i.e. $BS^2 < t_0\beta/64$;
memory-bound: BN / ReLU / pool (intensity < 1 FLOP/byte) leave the floor first, T grows linearly in BS^2 with slope $\sim 532/\beta$;
compute-bound: the convolutions (intensity 50-270) dominate, $T \rightarrow 17\,712\,BS^2/\pi$.

Baseline for comparison: additive model $T = Nt_0 + \text{Bytes}/\beta + \text{FLOPs}/\pi$ (no overlap at all).

θ is fitted by least squares on $\log(T_{\text{pred}}/T_{\text{meas}})$ over the base grid only; random (S, B) are held out.

5. Energy (joules per forward pass, whole board, NVML energy counter over a loop of synchronised passes).

Assumptions: board power = static part P_0 (idle + clocks, drawn for the whole duration of the pass) +
dynamic part proportional to the work: a fixed energy per FLOP and per byte moved; no DVFS model
(clocks are whatever the driver gives at that load); power-limit throttling not modelled.

$$\mathbf{E}(\mathbf{S}, \mathbf{B}; \boldsymbol{\theta}_E) = P_0 \mathbf{T}(\mathbf{S}, \mathbf{B}; \boldsymbol{\theta}) + e_F \text{FLOPs}(\mathbf{S}, \mathbf{B}) + e_B \text{Bytes}(\mathbf{S}, \mathbf{B}), \quad \boldsymbol{\theta}_E = (P_0, e_F, e_B)$$

P_0 [W], e_F [J/FLOP], e_B [J/byte] ≥ 0 by non-negative least squares (rows weighted by $1/E$) on the base grid.

Identifiability: for one fixed network $\text{FLOPs}/\text{Bytes} \approx 33$ for every (S, B) , so e_F and e_B are
collinear - only $e_F + e_B/33$ can be measured. We therefore fit (P_0, e_F) and set $e_B = 0$ (e_F then includes
the DRAM energy that accompanies each FLOP).

Consequences: energy per image $E/B = P_0 T/B + e_F \cdot 17793 S^2 + \dots$ - the static term is amortised by
batching in the launch-bound regime and vanishes once $T \propto B$; mean power $\bar{P} = E/T = P_0 + e_F \text{FLOPs}/T$
rises from P_0 (launch-bound) to $P_0 + e_F \pi$ (compute-bound), which must stay below the board power limit.